

TJSC 2026 - Analista de Sistemas

Engenharia de Software e Desenvolvimento

Aula 14 - Integração contínua e entrega contínua - CI/CD

Escopo da aula: Esta aula cobre o item 1.17 do edital: Integração contínua e entrega contínua - CI/CD. O objetivo é transformar prática de esteiras em definição de prova FGV, distinguindo Continuous Integration, Continuous Delivery e Continuous Deployment.

Padrão editorial: Material original, com teoria progressiva, exemplos de Java/Quarkus, Angular/TypeScript e sistemas corporativos, 20 questões autorais em estilo FGV, gabarito separado e comentários completos.

1. Identificação da aula e recorte do edital

Elemento	Definição
Aula	Aula 14 - Integração contínua e entrega contínua - CI/CD.
Trilha	Aula 14 da trilha de Engenharia de Software e Desenvolvimento.
Item do edital	1.17 Integração contínua e entrega contínua - CI/CD.
Recorte	Conceitos, pipeline, build, testes automatizados, quality gates, artefatos, ambientes, deploy, release, rollback, aprovação manual, entrega contínua e implantação contínua.
Fora do escopo principal	DevOps amplo, observabilidade, SRE e segurança de aplicações entram apenas quando ajudam a entender CI/CD, sem desviar para outras disciplinas.

A FGV tende a cobrar CI/CD em questões de engenharia de software, DevOps e arquitetura de entrega, normalmente exigindo que o candidato diferencie prática, ferramenta e resultado. Para a prova, não basta saber que existe Jenkins, GitHub Actions ou GitLab CI: é preciso reconhecer o que cada etapa da esteira faz e qual afirmação exagera o conceito.

No seu trabalho com Java, Quarkus, Angular, TypeScript, APIs, Git e esteiras corporativas, você provavelmente já viu pipelines de build, testes, deploy em homologação e promoção para produção. A prova costuma abstrair essa vivência: troca nomes reais por conceitos como integração contínua, entrega contínua, implantação contínua, pipeline como código, artefato versionado e portão de qualidade.

2. Siglas e termos essenciais

Sigla ou termo	Expansão	Sentido de prova
CI	Continuous Integration - Integração Contínua	Prática de integrar alterações frequentemente em uma linha principal, verificando-as por build e testes automatizados.
CD	Continuous Delivery - Entrega Contínua	Prática de manter o software sempre em condição de ser liberado, com automação até ambientes e eventual aprovação manual para produção.
CD	Continuous Deployment - Implantação Contínua	Variante em que toda alteração aprovada pela esteira é implantada automaticamente em produção, sem aprovação manual final.
Pipeline	Esteira de execução	Sequência automatizada de etapas como build, teste, análise, empacotamento, publicação e deploy.
IaC	Infrastructure as Code - Infraestrutura como Código	Gestão declarativa ou versionada de infraestrutura; pode apoiar CI/CD, mas não é sinônimo de CI/CD.
SCM	Source Code Management - Gerência de Código-Fonte	Controle de versão do código e gatilho comum para pipelines.
Artifact	Artefato	Resultado versionável do build: JAR, WAR, imagem de contêiner, pacote NPM, binário, arquivo ZIP etc.
Quality Gate	Portão de qualidade	Critério que bloqueia avanço se testes, cobertura, análise estática, vulnerabilidades ou políticas não forem atendidas.
Rollback	Retorno a versão anterior	Mecanismo para restaurar versão estável após falha; não substitui testes, mas reduz impacto.

3. Ideia central: da integração manual à esteira automatizada

Integração contínua e entrega contínua surgem para reduzir o risco clássico de integração tardia: várias pessoas desenvolvem por dias ou semanas em ramos separados, juntam tudo perto da entrega, descobrem conflitos, testes quebrados, dependências incompatíveis e defeitos de ambiente. A prática moderna tenta tornar a integração frequente, pequena, automatizada e verificável.

Integração contínua não é “apertar um botão no Jenkins”. A ferramenta apenas executa a prática. A essência está em integrar mudanças pequenas com frequência, acionar build automatizado, executar testes e descobrir falhas rapidamente. A entrega contínua amplia isso: depois de validado, o software é empacotado e fica pronto para ir a um ambiente de uso, inclusive produção, com baixo esforço operacional.

Ponto FGV: Quando a alternativa diz que CI/CD “elimina” erros, conflitos ou necessidade de testes manuais, desconfie. CI/CD reduz risco e acelera feedback, mas não garante ausência de defeitos nem substitui governança.

4. Integração Contínua - Continuous Integration

Continuous Integration - Integração Contínua (CI) é a prática pela qual os desenvolvedores integram frequentemente suas alterações a uma linha principal do código, normalmente com execução automática de build e testes. A finalidade é detectar problemas de integração cedo, enquanto a alteração ainda é pequena e compreensível.

Na linguagem de prova, CI envolve três ideias inseparáveis: **integração frequente**, **verificação automatizada** e **feedback rápido**. Sem automação de build e testes, há apenas integração frequente manual. Sem integração frequente, há apenas automação eventual. Sem feedback, a equipe descobre tarde que algo quebrou.

Elemento de CI	O que significa	Exemplo no seu contexto
Commit pequeno	Mudança reduzida, revisável e integrável.	Alterar um endpoint Quarkus e o respectivo teste unitário.
Build automatizado	Compilação/empacotamento acionado automaticamente.	mvn clean verify para Java/Quarkus; npm ci && npm run build para Angular.
Testes automatizados	Verificações executadas sem intervenção humana.	JUnit, Mockito, RestAssured, Karma/Jasmine, Jest, Cypress.
Feedback rápido	Falha visível logo após commit ou merge request.	Pipeline vermelho no merge request antes de entrar na main.
Linha principal saudável	Branch principal deve permanecer integrável.	main/master/develop protegido com quality gate.

A CI não exige uma ferramenta específica. Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps, Bamboo ou soluções corporativas internas podem implementar a prática. A FGV costuma usar nomes de ferramentas para testar se você confunde tecnologia com conceito.

5. Entrega Contínua versus Implantação Contínua

O ponto que mais derruba candidato é a diferença entre Continuous Delivery e Continuous Deployment. Em português, ambos aparecem como “CD”, mas não significam a mesma coisa.

Conceito	Pergunta-chave	Produção é automática?	Como a FGV pode cobrar
Continuous Delivery - Entrega	O software está sempre pronto	Não necessariamente. Pode	Ambiente de homologação

Conceito	Pergunta-chave	Produção é automática?	Como a FGV pode cobrar
Contínua	para liberar?	haver aprovação manual para produção.	automatizado, pacote pronto e botão/aprovação para produção.
Continuous Deployment - Implantação Contínua	Toda alteração aprovada pela esteira vai automaticamente para produção?	Sim, se passar todos os critérios.	Sem aprovação manual final; deploy automático em produção após testes e gates.

Entrega contínua é mais comum em ambientes regulados, bancários e públicos porque permite automação forte sem abrir mão de janelas, aprovações, segregação de funções e ritos de mudança. Implantação contínua é mais agressiva: se o pipeline aprova, a produção recebe automaticamente.

Memorização útil: Delivery = pronto para entregar; Deployment = implantado de fato. A palavra “deployment” aponta para colocar em ambiente, especialmente produção quando o contexto disser deploy automático final.

6. Anatomia de uma pipeline de CI/CD

Pipeline é a **sequência organizada de etapas executadas por uma ferramenta de automação**. Em prova, pipeline não é sinônimo de deploy: deploy é uma das possíveis etapas. Uma pipeline completa pode começar no commit e terminar no ambiente de produção ou apenas gerar um artefato pronto para liberação.

Etapas	Finalidade	Erro comum em prova
Checkout	Obter código-fonte do repositório.	Confundir checkout com deploy.
Build	Compilar, resolver dependências e empacotar.	Achar que build testa regra de negócio por si só.
Test	Executar testes unitários, integração, sistema, contrato ou E2E conforme o estágio.	Dizer que teste manual é o centro de CI.
Static analysis	Verificar qualidade, estilo, bugs prováveis, vulnerabilidades e cobertura.	Tratar análise estática como teste funcional.
Package	Gerar artefato imutável e rastreável.	Recompilar em cada ambiente, perdendo reprodutibilidade.
Publish artifact	Publicar JAR, imagem Docker, pacote NPM ou outro artefato em repositório.	Confundir artefato com código-fonte.
Deploy to environment	Implantar em desenvolvimento, teste, homologação, staging ou produção.	Chamar qualquer deploy de implantação contínua.
Approval/Change gate	Exigir aprovação humana ou critério de governança.	Achar que aprovação manual impede entrega contínua.
Smoke test/health check	Checar se o serviço subiu e responde minimamente.	Confundir smoke test com suíte completa de regressão.

7. Pipeline como código - Pipeline as Code

Pipeline as Code - pipeline como código - é a prática de descrever a esteira em arquivo versionado junto ao projeto ou em repositório controlado. Isso aumenta rastreabilidade, revisão, reuso e auditoria. A FGV gosta de cobrar esse ponto com Jenkinsfile, YAML de GitHub Actions ou .gitlab-ci.yml.

```
# Exemplo simplificado de GitHub Actions para Java/Quarkus
name: build-test
```

```

on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - run: ./mvnw clean verify

```

No exemplo, o workflow é acionado por push e pull request; o job executa em runner Ubuntu; as steps fazem checkout, configuram Java 17 e executam Maven. Isso representa CI. Só seria CD se houvesse empacotamento/publicação/deploy; e só seria implantação contínua se o deploy final em produção fosse automático após passar pelos critérios.

```

// Exemplo simplificado de Jenkinsfile declarativo
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh './mvnw clean package' } }
    stage('Test') { steps { sh './mvnw test' } }
    stage('Deploy homolog') { steps { sh './scripts/deploy-hml.sh' } }
  }
}

```

Jenkinsfile é arquivo textual que define a pipeline e pode ser versionado no controle de versão. O ganho de prova é perceber que isso favorece rastreabilidade e revisão do processo de entrega, mas não garante, sozinho, qualidade, segurança ou ausência de falhas.

8. Artefato, ambiente e reprodutibilidade

Uma boa pipeline evita recompilar a aplicação de modo diferente em cada ambiente. O padrão mais robusto é construir uma vez, gerar um artefato versionado e promover esse mesmo artefato por ambientes. Isso reduz o clássico problema “funcionou em homologação, mas a versão de produção foi recompilada com dependência diferente”.

Conceito	Definição	Exemplo
Artefato imutável	Pacote gerado pela esteira e não alterado manualmente.	app-1.4.2.jar; imagem Docker com digest; pacote frontend versionado.
Repositório de artefatos	Local para armazenar artefatos versionados.	Nexus, Artifactory, GitHub Packages, GitLab Registry, registry corporativo.
Promoção de ambiente	Mover o mesmo artefato para ambientes sucessivos.	dev -> teste -> homologação -> produção.
Configuração externa	Parâmetros de ambiente fora do binário.	URL de API, credenciais via secret, perfil de Quarkus por ambiente.

Pegadinha: Pipeline madura não significa “editar configuração dentro do pacote antes de cada deploy”. Configuração deve ser tratada como dado de ambiente, secret ou variável controlada, preservando a rastreabilidade do artefato.

9. Ambientes, aprovações e governança

Em sistemas públicos, bancários e corporativos, **CI/CD precisa conviver com segregação de funções, janelas de mudança, auditoria, controles de acesso, rastreabilidade e aprovação**. Isso não nega a entrega contínua. Uma esteira pode automatizar build, testes, empacotamento, deploy em homologação e deixar a produção dependente de aprovação humana.

A FGV costuma usar frases absolutas como “entrega contínua elimina aprovações” ou “implantação contínua exige sempre deploy manual”. Ambas são perigosas. Entrega contínua admite aprovação manual final; implantação contínua, por definição de prática, tende a automatizar a ida a produção se os critérios forem satisfeitos.

Prática	Finalidade	Risco se mal feita
Branch protection	Impedir merge sem revisão, testes ou status checks.	Burlar gates para “subir rápido”.
Manual approval	Controlar promoção para produção.	Virar gargalo se todo passo for manual.
Segregação de funções	Separar quem desenvolve, aprova e opera mudanças críticas.	Pipeline com credenciais amplas demais nas mãos erradas.
Auditoria de execução	Registrar quem aprovou, qual commit, qual artefato e quando foi implantado.	Não conseguir rastrear incidente ou regressão.

10. Quality gates e pirâmide de validação na esteira

Quality gate é uma condição que precisa ser satisfeita para a pipeline avançar. Pode envolver testes unitários, testes de integração, cobertura mínima, análise estática, ausência de vulnerabilidade crítica, aprovação de revisão, assinatura de artefato, geração de SBOM - Software Bill of Materials - ou checagem de licença.

Em prova, quality gate não é a mesma coisa que teste unitário. Teste unitário pode ser um insumo do gate. O gate é o critério de decisão: passa ou bloqueia.

Gate	O que mede	Pegadinha
Testes unitários	Comportamento de unidades isoladas.	Cobertura alta não prova ausência de defeitos.
Testes de integração	Interação com banco, fila, API ou componente externo controlado.	Não são iguais a testes de sistema completos.
Análise estática	Padrões, complexidade, bugs prováveis, vulnerabilidades conhecidas.	Não executa a regra de negócio.
Cobertura	Percentual de código exercitado por testes.	Cobertura mede execução, não qualidade das asserções.
SCA	Software Composition Analysis - análise de dependências.	Não substitui teste funcional.

11. Estratégias de implantação e liberação

Deploy e release são conceitos próximos, mas não idênticos. Deploy é colocar uma versão em ambiente. Release é disponibilizar funcionalidade para usuários. Com feature toggles, pode-se fazer deploy de código sem liberar a funcionalidade para todos.

Estratégia	Como funciona	Vantagem	Cuidado
Rolling update	Substitui instâncias gradualmente.	Menor indisponibilidade.	Conviver com duas versões temporariamente.
Blue-green deployment	Mantém dois ambientes semelhantes; alterna tráfego do azul para o verde.	Rollback rápido por troca de tráfego.	Custo de manter ambientes duplicados.
Canary release	Libera para pequena parcela de usuários/instâncias e amplia se métricas forem boas.	Reduz impacto de falha.	Exige observabilidade e segmentação.
Feature toggle	Funcionalidade ativada/desativada por configuração.	Separa deploy de release.	Toggles antigos acumulam dívida técnica.
Rollback	Retorna a versão anterior conhecida como estável.	Mitiga incidente.	Pode falhar se houve migração incompatível de dados.

12. Segurança e segredos em CI/CD

Pipeline também é superfície de ataque. Tokens, chaves, certificados, usuários de deploy e variáveis sensíveis não devem ser gravados no repositório nem impressos em log. O ideal é usar secret manager, permissões mínimas, rotação de credenciais, revisão de dependências, assinatura de artefatos e controle de quem pode aprovar deploy.

Na sua realidade de Banco do Brasil e sistemas corporativos, esse ponto aparece como credenciais protegidas, esteiras segregadas, ambientes controlados e rastreabilidade. Na FGV, aparece como alternativa que tenta vender “automatização” como dispensa de controle. CI/CD bom automatiza com controle; não automatiza o descontrole.

- Não guardar senha no Jenkinsfile, YAML ou script versionado.
- Não usar token com privilégio de administrador quando um token restrito bastaria.
- Não expor variáveis sensíveis em logs de build.
- Não publicar artefato sem saber commit, versão e dependências associadas.
- Não confundir velocidade de deploy com ausência de governança.

13. CI/CD em Java/Quarkus e Angular/TypeScript

Um cenário plausível para seu trabalho: backend Quarkus e frontend Angular. A pipeline pode ter jobs separados para backend e frontend, usar cache de dependências, executar testes, gerar artefatos e publicar imagem de contêiner. O ponto de prova é separar o que é build, teste, empacotamento, publicação e deploy.

```
# Backend Quarkus - etapas típicas
./mvnw clean verify
./mvnw package -DskipTests=false
docker build -t registry.interno/projeto-api:${COMMIT_SHA} .
docker push registry.interno/projeto-api:${COMMIT_SHA}
```

```
# Frontend Angular - etapas típicas
npm ci
npm run lint
```

```
npm test -- --watch=false
npm run build -- --configuration production
```

Observe que npm ci é preferível a npm install em pipeline porque instala conforme o lockfile, aumentando reprodutibilidade. Em Java, o uso de Maven Wrapper ou Gradle Wrapper ajuda a padronizar versão da ferramenta. Em Quarkus, perfis e variáveis de ambiente ajudam a separar configuração de ambiente do artefato.

14. Diferenças entre prática real e cobrança FGV

Na prática corporativa	Na prova FGV
A equipe chama toda esteira de "deploy".	A banca distingue build, teste, entrega, implantação, release e rollback.
Há aprovações, janelas e ritos de mudança.	Isso pode continuar sendo entrega contínua se o software estiver pronto para liberação.
Muitos projetos têm pipelines parcialmente manuais.	A banca procura a definição ideal da prática, não o apelido usado na empresa.
Ferramentas variam: Jenkins, GitLab, GitHub, Azure DevOps, esteiras internas.	A ferramenta é meio; o conceito é integração/entrega automatizada e rastreável.
Testes podem ser insuficientes ou lentos.	CI pressupõe build/testes automatizados e feedback rápido; flaky test é problema da esteira.
Produção pode exigir aprovação humana.	Isso diferencia delivery de deployment automático.

15. Pegadinhas FGV de alto ROI

- CI não é a mesma coisa que controle de versão. Git/SVN apoiam CI, mas não são CI.
- CI não é só compilar. Build sem teste automatizado reduz muito o valor da integração contínua.
- Continuous Delivery não exige deploy automático em produção; Continuous Deployment exige automação do deploy final, se a mudança passar na esteira.
- Pipeline as Code aumenta rastreabilidade e revisão, mas não garante qualidade por si só.
- Quality gate é critério de bloqueio; teste é uma das evidências usadas pelo gate.
- Deploy não é necessariamente release; feature toggle pode separar implantação de disponibilização ao usuário.
- Rollback reduz impacto, mas não substitui testes e pode ser complexo quando há migração de banco incompatível.
- Automação não elimina governança; em contexto público/corporativo, CI/CD deve conviver com auditoria e segregação de funções.
- Ferramentas proprietárias e open source podem implementar CI/CD; a banca pode cobrar benefícios, não marketing de ferramenta.
- Pipeline quebrada deve interromper avanço; ignorar teste quebrado para entregar mais rápido contraria a lógica da CI.

16. Questões autorais no padrão FGV

A seguir, resolva as 20 questões sem consultar o gabarito. As questões foram elaboradas para simular o estilo da FGV: cenários de TI, alternativas próximas, comandos de correta/incorreta, V/F, afirmativas, relação de conceitos, código e interpretação de pipeline.

1. Em uma equipe responsável por APIs internas de um tribunal, os desenvolvedores passaram a integrar pequenas alterações ao ramo principal do repositório várias vezes ao dia. A cada integração, uma esteira

executa compilação, testes automatizados e análise básica de qualidade, bloqueando a alteração quando há falha. No vocabulário de engenharia de software, essa prática é mais bem caracterizada como

- (A) entrega contínua, pois qualquer execução de testes em pipeline equivale a disponibilizar software em produção.
- (B) implantação contínua, pois a existência de uma esteira automatizada elimina a necessidade de aprovação humana.
- (C) integração contínua, pois as mudanças são incorporadas frequentemente e verificadas por build e testes automatizados.
- (D) gerência de configuração, pois o objetivo principal é apenas catalogar versões e itens de configuração.
- (E) orquestração de contêineres, pois a automação de compilação depende necessariamente de Kubernetes.

2. Uma aplicação Java/Quarkus é compilada, testada, empacotada em imagem de contêiner e implantada automaticamente em ambiente de homologação a cada merge aprovado. A promoção para produção depende de aprovação formal do gestor de mudança. Considerando a distinção clássica entre Continuous Delivery e Continuous Deployment, o cenário descreve principalmente

- (A) integração contínua apenas, porque qualquer aprovação manual impede que exista entrega contínua.
- (B) entrega contínua, pois o software fica pronto para liberação e a produção pode depender de aprovação manual.
- (C) implantação contínua, pois todo deploy em homologação equivale a deploy automático em produção.
- (D) controle de versão centralizado, pois a aprovação formal substitui o pipeline.
- (E) teste de sistema, pois o elemento determinante é a existência de um ambiente de homologação.

3. Com relação a pipelines como código em ferramentas de CI/CD, analise as afirmativas a seguir. I. Um Jenkinsfile ou arquivo YAML de pipeline pode ser versionado junto ao código, favorecendo revisão, rastreabilidade e auditoria. II. Pipeline como código garante, por si só, que o software produzido esteja livre de vulnerabilidades e defeitos funcionais. III. A descrição da pipeline em arquivo textual permite reproduzir e evoluir o processo de build, teste e deploy de modo controlado. Está correto o que se afirma em

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

4. Considere as etapas a seguir, em uma esteira de entrega de uma aplicação web: 1. Compilar o backend e gerar pacote executável. 2. Executar testes unitários e de integração. 3. Publicar a imagem versionada no registry corporativo. 4. Colocar a versão aprovada no ambiente de homologação. As etapas descritas correspondem, respectivamente, a

- (A) deploy, release, checkout e rollback.
- (B) build, testes, publicação de artefato e deploy.
- (C) merge, fork, pull request e tag.
- (D) release, rollback, merge e build.
- (E) provisionamento, observabilidade, autenticação e autorização.

5. Assinale a afirmativa INCORRETA sobre integração contínua, entrega contínua e implantação contínua.

- (A) A integração contínua busca reduzir problemas de integração tardia por meio de integrações frequentes e verificações automatizadas.
- (B) A entrega contínua pressupõe que o software possa ser liberado de forma confiável, mesmo que a produção ainda exija uma autorização humana.
- (C) A implantação contínua automatiza a ida para produção quando as etapas e critérios da pipeline são satisfeitos.
- (D) A existência de Git, por si só, caracteriza uma organização como praticante de integração contínua, ainda que não haja build ou testes automatizados.
- (E) A automação de testes e build é elemento relevante para obter feedback rápido em uma pipeline de CI.

6. Um repositório possui o seguinte trecho de workflow: name: verify; on: [push, pull_request]; jobs: test: runs-on: ubuntu-latest; steps: checkout, setup-java 17, ./mvnw clean verify. A leitura mais adequada desse trecho é a de que ele

- (A) define uma política de autorização de usuários no ambiente de produção.
- (B) executa uma rotina típica de integração contínua, acionada por eventos do repositório.
- (C) realiza implantação contínua em produção, pois todo workflow YAML implica deploy final.
- (D) substitui a necessidade de testes automatizados, pois o comando verify apenas compila.
- (E) representa uma estratégia canary, pois usa Java 17 em ambiente Ubuntu.

7. Em uma equipe Angular/TypeScript, a pipeline executa npm ci, lint, testes automatizados e build de produção. Em determinado commit, os testes quebram, mas o líder decide ignorar a falha e promover o pacote para homologação “para ganhar tempo”. À luz das boas práticas de CI/CD, essa decisão

- (A) é compatível com integração contínua, pois a falha foi identificada e pode ser corrigida depois.
- (B) é obrigatória em entrega contínua, pois a entrega deve sempre priorizar velocidade sobre qualidade.
- (C) contraria a lógica de feedback e bloqueio da esteira, pois falhas em testes relevantes devem impedir avanço do artefato.
- (D) é uma forma de implantação contínua, pois torna o deploy mais frequente.
- (E) é requisito de DevOps, pois elimina a separação entre desenvolvimento e operação.

8. Em uma política de release de sistemas corporativos, a funcionalidade é implantada em produção, mas permanece desativada para usuários finais até que uma chave de configuração seja habilitada para grupos específicos. Esse mecanismo é mais bem associado a

- (A) feature toggle, pois separa deploy técnico da liberação funcional.
- (B) checkout, pois alterna o ramo principal do repositório.
- (C) build determinístico, pois recompila a aplicação para cada usuário.
- (D) file locking, pois impede concorrência de edição no código.
- (E) integração manual, pois substitui automação por decisão humana.

9. Com relação ao uso de Jenkinsfile em esteiras de CI/CD, assinale V para a afirmativa verdadeira e F para a falsa. () Permite registrar a definição da pipeline como arquivo textual versionável. () Dispensa o uso de repositório de código-fonte, pois a pipeline passa a armazenar todo o sistema. () Pode favorecer auditoria e revisão das mudanças feitas no processo de build, teste e deploy. A sequência correta é

- (A) V - F - V.
- (B) V - V - F.
- (C) F - V - V.
- (D) F - F - V.
- (E) V - F - F.

10. Em uma pipeline, após a etapa de build, é gerado um arquivo app-2.3.1.jar e uma imagem registry.local/app@sha256:abc123. Esses itens são armazenados em repositório próprio e promovidos para os ambientes seguintes sem recompilação. O principal benefício dessa abordagem é

- (A) eliminar a necessidade de controle de versão do código-fonte.
- (B) garantir que a mesma unidade construída e testada seja rastreada e promovida entre ambientes.
- (C) permitir que cada ambiente altere livremente o binário para adaptar regras de negócio.
- (D) impedir qualquer falha em produção, inclusive defeitos de requisito.
- (E) substituir testes de integração por versionamento semântico.

11. Uma aplicação é disponibilizada em duas infraestruturas equivalentes: a versão atual atende no ambiente azul, enquanto a nova versão é implantada no ambiente verde. Após validações, o tráfego é redirecionado para o verde; se houver falha, o tráfego retorna ao azul. Essa estratégia é conhecida como

- (A) rolling update.
- (B) blue-green deployment.
- (C) canary release.
- (D) trunk-based development.
- (E) forking workflow.

12. Sobre DevOps e CI/CD, assinale a alternativa correta.

(A) DevOps é uma ferramenta específica de automação de pipeline, enquanto CI/CD é uma metodologia exclusivamente documental.

(B) CI/CD é um conjunto de práticas de automação de integração, entrega e implantação que pode materializar princípios DevOps, mas não esgota toda a cultura DevOps.

(C) DevOps recomenda separar completamente desenvolvimento e operação, evitando colaboração para preservar responsabilidades.

(D) CI/CD só é aplicável a microsserviços e não pode ser usado em sistemas monolíticos ou aplicações web tradicionais.

(E) Adoção de CI/CD elimina a necessidade de monitoramento, revisão de código e controle de mudanças.

13. Uma equipe percebe que a pipeline falha de maneira intermitente: o mesmo commit ora passa, ora falha, sem alteração no código. Esse fenômeno costuma ser chamado de teste flaky. Em termos de gestão da esteira, a postura mais adequada é

(A) remover todos os testes automatizados, pois testes instáveis provam que automação não funciona.

(B) ignorar falhas intermitentes e manter deploy automático para produção em todos os casos.

(C) tratar a instabilidade como débito da pipeline, investigando isolamento, dependências externas, dados de teste, tempo e concorrência.

(D) substituir controle de versão por cópia manual do código.

(E) classificar a falha como rollback, independentemente de ter havido implantação.

14. Em determinado órgão público, o mesmo artefato aprovado é implantado sucessivamente em desenvolvimento, teste, homologação e produção, variando apenas parâmetros externos e secrets por ambiente. Essa prática está mais alinhada ao princípio de

(A) imutabilidade e promoção controlada de artefatos.

(B) recompilação manual em cada ambiente.

(C) exposição de credenciais em repositório para facilitar auditoria.

(D) substituição de testes por aprovação verbal.

(E) acoplamento entre binário e configuração de produção.

15. Analise as afirmativas sobre segurança em pipelines de CI/CD. I. Secrets de produção devem ser protegidos por mecanismos próprios, e não gravados em texto claro no YAML da pipeline. II. A pipeline deve preferir permissões mínimas necessárias para executar build, publicação e deploy. III. Logs de build são local apropriado para imprimir tokens, pois isso facilita auditoria pública. Está correto o que se afirma em

(A) I, apenas.

(B) I e II, apenas.

(C) II e III, apenas.

(D) III, apenas.

(E) I, II e III.

16. Relacione os conceitos às descrições. 1. Quality gate. 2. Artifact repository. 3. Smoke test. 4. Rollback. () Armazena pacotes, imagens ou binários gerados pela esteira. () Critério que bloqueia avanço quando condições de qualidade não são atendidas. () Verificação rápida após implantação para confirmar funcionamento básico. () Retorno a versão anterior conhecida como estável. A sequência correta é

(A) 2 - 1 - 3 - 4.

(B) 1 - 2 - 4 - 3.

(C) 3 - 4 - 1 - 2.

(D) 2 - 3 - 1 - 4.

(E) 4 - 1 - 2 - 3.

17. Uma pipeline de backend Java executa: ./mvnw clean verify; docker build; docker push; deploy-hml.sh; aguarda aprovação; deploy-prod.sh. Considerando a classificação de CI/CD, a descrição mais precisa é

(A) apenas controle de versão, pois usa comandos em shell.

(B) integração contínua apenas, pois contém testes.

- (C) entrega contínua com aprovação manual antes de produção.
- (D) implantação contínua plena, pois qualquer deploy em homologação implica produção automática.
- (E) ausência de CI/CD, pois Maven e Docker não podem aparecer na mesma esteira.

18. Em uma pipeline de frontend Angular, a etapa npm ci falha porque o package-lock.json está inconsistente com o package.json. A interpretação mais adequada é que

- (A) a falha é desejável como feedback, pois a pipeline preserva reprodutibilidade das dependências em vez de instalar versões arbitrárias.
- (B) a etapa deveria ser substituída por edição manual em produção, pois frontend não exige build reprodutível.
- (C) o erro prova que integração contínua só funciona para Java e não para TypeScript.
- (D) a falha deve ser ignorada, pois lockfile não tem relação com confiabilidade de entrega.
- (E) o pipeline converteu automaticamente entrega contínua em implantação contínua.

19. No contexto de integração contínua, ramos de longa duração e integração tardia tendem a aumentar

- (A) a frequência de feedback rápido e a simplicidade da integração.
- (B) a rastreabilidade dos artefatos e a qualidade dos testes unitários.
- (C) o risco de conflitos, divergências e descoberta tardia de defeitos de integração.
- (D) a garantia de ausência de defeitos em produção.
- (E) a eliminação da necessidade de revisão de código.

20. Considere o resumo de uma execução de pipeline: checkout: sucesso; build: sucesso; testes unitários: sucesso; análise estática: falha por vulnerabilidade crítica; publicação de artefato: não executada; deploy: não executado. A conclusão mais adequada é

- (A) a pipeline se comportou de forma compatível com quality gate, bloqueando etapas posteriores diante de falha crítica.
- (B) a pipeline concluiu implantação contínua, pois checkout e build foram bem-sucedidos.
- (C) o artefato deve ser publicado mesmo com vulnerabilidade crítica, pois análise estática é apenas documentação.
- (D) o deploy não executado caracteriza rollback automático, ainda que nenhuma versão tenha sido implantada.
- (E) a falha na análise estática comprova erro no controle de versão, não na qualidade do código ou dependências.

17. Gabarito resumido

1-C; 2-B; 3-C; 4-B; 5-D; 6-B; 7-C; 8-A; 9-A; 10-B; 11-B; 12-B; 13-C; 14-A; 15-B; 16-A; 17-C; 18-A; 19-C; 20-A.

18. Comentários completos

1. Gabarito: C. A alternativa C está correta porque descreve a essência da Continuous Integration - Integração Contínua: mudanças frequentes na linha principal, verificadas por build e testes automatizados. A alternativa A confunde CI com entrega contínua; testar em pipeline não significa disponibilizar para produção. A B confunde automação com implantação contínua; não há deploy automático em produção no enunciado. A D usa um conceito verdadeiro, gerência de configuração, mas estreito demais para o cenário. A E desloca o tema para Kubernetes, ferramenta/infraestrutura que não é requisito para CI.

2. Gabarito: B. A alternativa B está correta porque o artefato é preparado e implantado em homologação automaticamente, ficando pronto para liberação, mas produção depende de aprovação manual; isso é compatível com Continuous Delivery - Entrega Contínua. A A erra ao dizer que aprovação manual impede delivery. A C confunde homologação com produção automática. A D troca o assunto por controle de versão centralizado. A E foca no ambiente de homologação, mas ignora o fluxo de entrega.

3. Gabarito: C. I e III estão corretas: pipeline como código, por exemplo Jenkinsfile ou YAML, favorece versionamento, revisão, rastreabilidade e evolução controlada da esteira. II está errada porque arquivo de pipeline não garante ausência de vulnerabilidades ou defeitos; ele apenas descreve automações. A A ignora a III, que também está correta. A B e D dependem da II, falsa. A E aceita a falsa garantia absoluta.

4. Gabarito: B. A sequência correta é build, testes, publicação de artefato e deploy. A A usa termos reais, mas fora de ordem e com sentido errado. A C mistura operações de Git que não correspondem às etapas descritas. A D inverte release, rollback e build. A E traz conceitos de infraestrutura e segurança que não representam diretamente as etapas do enunciado.

5. Gabarito: D. A alternativa D é a incorreta: Git é controle de versão e pode apoiar CI, mas não caracteriza integração contínua sozinho. É preciso integração frequente, build e testes/validações automatizados. A A descreve corretamente o objetivo da CI. A B acerta que delivery pode ter aprovação manual final. A C define deployment contínuo. A E acerta a importância de automação para feedback rápido.

6. Gabarito: B. O workflow acionado por push e pull request, com checkout, Java 17 e mvnw clean verify, é típico de CI. A A confunde workflow com autorização de produção. A C erra ao dizer que todo YAML implica deploy final. A D erra porque verify normalmente compila e executa verificações/testes no ciclo Maven. A E usa canary sem qualquer indício de liberação gradual.

7. Gabarito: C. A alternativa C está correta: se testes relevantes falham, a esteira deve bloquear avanço ou ao menos impedir promoção como se estivesse saudável. A A é tentadora porque a falha foi detectada, mas ignorá-la anula o feedback. A B absolutiza velocidade, contrariando a qualidade. A D confunde frequência de deploy com implantação contínua correta. A E distorce DevOps, que aumenta colaboração, mas não elimina controles de qualidade.

8. Gabarito: A. Feature toggle é o mecanismo que separa deploy de release: o código está implantado, mas a funcionalidade pode ficar desligada ou restrita a grupos. B confunde com operação de controle de versão. C inventa recompilação por usuário. D trata de bloqueio de arquivo, típico de sistemas centralizados. E confunde decisão de liberação com ausência de automação.

9. Gabarito: A. A sequência é V - F - V. Jenkinsfile pode ser versionado, revisado e auditado. Ele não dispensa repositório de código-fonte; pelo contrário, normalmente é armazenado nele ou em SCM. Também favorece auditoria do processo de entrega. As demais alternativas erram a segunda afirmativa ou negam uma das verdadeiras.

10. Gabarito: B. A alternativa B está correta porque promover o mesmo artefato aumenta rastreabilidade e reprodutibilidade: aquilo que foi testado é aquilo que segue adiante. A A erra porque controle de versão continua essencial. A C é o oposto do desejável, pois alteração livre do binário rompe rastreabilidade. A D promete garantia absoluta. A E confunde artefato com substituição de testes de integração.

11. Gabarito: B. Blue-green deployment usa dois ambientes semelhantes, alternando tráfego entre versão atual e nova. A A rolling update substitui instâncias gradualmente no mesmo conjunto. A C canary libera para parcela pequena de usuários/instâncias. A D trata de estratégia de ramificação. A E é fluxo de colaboração em Git, não estratégia de deploy.

12. Gabarito: B. CI/CD materializa práticas de automação que combinam bem com DevOps, mas DevOps é mais amplo: cultura, colaboração, fluxo, automação, feedback e melhoria contínua. A A transforma DevOps em ferramenta. A C diz o oposto de DevOps, que busca colaboração entre desenvolvimento e operação. A D restringe indevidamente a microsserviços. A E promete eliminação indevida de monitoramento, revisão e controle.

13. Gabarito: C. Teste flaky é problema sério da confiabilidade da pipeline; deve ser investigado com foco em isolamento, dependências, tempo, concorrência e dados. A A joga fora a automação por causa de um problema corrigível. A B normaliza risco. A D não tem relação. A E confunde falha de teste com rollback, que pressupõe retorno após implantação.

14. Gabarito: A. Promover o mesmo artefato por ambientes, alterando apenas parâmetros externos e secrets, preserva imutabilidade e rastreabilidade. A B descreve prática menos reprodutível. A C é insegura, pois secrets não devem estar em repositório. A D substitui indevidamente validações por fala informal. A E acopla binário à configuração de produção, reduzindo portabilidade.

15. Gabarito: B. I e II estão corretas: secrets devem ser protegidos e permissões devem seguir o menor privilégio. III está errada porque logs não são local apropriado para expor tokens; isso amplia risco de vazamento. A A ignora a II. C e D aceitam a III. E aceita todas, incluindo a falsa.

16. Gabarito: A. Artifact repository armazena artefatos; quality gate bloqueia avanço por critérios de qualidade; smoke test é verificação rápida pós-deploy; rollback retorna versão anterior. As demais sequências trocam critério por armazenamento, smoke por rollback ou atribuem o retorno de versão a conceitos que não fazem esse papel.

17. Gabarito: C. A pipeline faz build/testes, publica imagem, faz deploy em homologação, aguarda aprovação e só então vai para produção. Isso é entrega contínua com aprovação manual antes de produção. A A reduz indevidamente tudo a controle de versão. A B ignora etapas de publicação e deploy. A D erra porque produção não é automática. A E inventa incompatibilidade entre Maven e Docker.

18. Gabarito: A. npm ci falhar por inconsistência de lockfile é feedback útil: a pipeline preserva instalação reprodutível. B é insegura e antitécnica. C restringe CI a Java, o que é falso. D despreza a importância do lockfile para dependências. E não tem relação: falha de dependência não transforma delivery em deployment.

19. Gabarito: C. Ramos longos e integração tardia aumentam risco de conflitos, divergências e descoberta tardia de defeitos. A A afirma o contrário do efeito esperado pela CI. B mistura rastreabilidade e qualidade de testes sem suporte. D promete ausência de defeitos. E remove indevidamente revisão de código.

20. Gabarito: A. A execução mostra um gate de qualidade: análise estática falhou por vulnerabilidade crítica e, por isso, publicação e deploy não seguiram. B confunde build com implantação. C desvaloriza análise estática, que pode compor critérios de bloqueio. D chama de rollback sem implantação anterior. E atribui a falha ao controle de versão, embora o problema indicado seja qualidade/segurança.

19. Checklist final do que memorizar

- CI = integrar frequentemente + build/testes automatizados + feedback rápido.
- Continuous Delivery = software sempre pronto para liberar; pode haver aprovação manual para produção.
- Continuous Deployment = deploy automático em produção se a esteira aprovar.
- Pipeline não é sinônimo de deploy: pipeline pode conter checkout, build, testes, análise, publicação, deploy e aprovação.
- Jenkinsfile/YAML = pipeline como código, com rastreabilidade e revisão, mas sem garantia absoluta de qualidade.
- Artefato deve ser rastreável, versionado e preferencialmente promovido entre ambientes sem recompilar.
- Quality gate é critério de bloqueio; testes e análises são evidências usadas pelo gate.
- Deploy não é necessariamente release: feature toggle pode separar implantação técnica da liberação ao usuário.
- Automação não elimina governança, auditoria, revisão, segregação de funções ou segurança.

20. Fontes técnicas e referências utilizadas

Edital TJSC 2026 retificado - conteúdo programático e distribuição de questões para Analista de Sistemas.

Manual Mestre de Calibração FGV para Aulas e Questões - régua editorial do projeto TJSC 2026.

FGV Conhecimento - provas oficiais correlatas de TI consultadas para calibrar formato de cobrança, vocabulário, densidade e padrões de alternativas.

Martin Fowler - Continuous Integration e Software Delivery Guide.

Jenkins Documentation - Pipeline, Jenkinsfile e Pipeline as Code.

GitHub Actions Documentation - workflows, jobs, steps e sintaxe YAML.

GitLab Documentation - CI/CD, pipelines, stages e jobs.

Atlassian Continuous Delivery - distinções entre continuous integration, delivery e deployment.

Documentação de Maven, Quarkus, Angular e práticas correntes de automação foram usadas apenas para exemplificação técnica, sem substituir o foco do edital.